

JARVIS is not a wrapper

The wrapper accusation has a clean form. *You forward user input to an LLM, you forward the LLM's response back, and the middle isn't load-bearing. The value is the LLM's, redistributed at a markup.*

I'll concede the easy version up front. Most "AI agents" *are* wrappers. Most "Claude-powered X" products are a system prompt, a chat-completion call, and a UI. If the underlying model regresses, the product regresses linearly. If the model gets cheaper, the product is repriced. The middle is decorative.

JARVIS is not that. The simplest test: would removing the LLM kill the system, or replace one substrate?

But first — what JARVIS actually is, because most people see one of two surfaces and assume the surface is the system.

What JARVIS actually is

You may have seen the Telegram bot (`@JarvisMind1828383bot`). You may have seen my GitHub commits. You may have seen the published essays. All of these are *outputs* of JARVIS. None of them is JARVIS.

JARVIS is the agent overlay architecture I run on top of Claude. Eight layers, all live, all producing the artifacts you see:

1. **Hook layer** — PreToolUse / SessionStart / Stop hooks intercepting every tool call, every session boot, every commit
2. **Persistence layer** — six distinct artifact classes persisted across sessions; ~389 memory files in the primary directory plus subdirectories for NDA-locked, archived, and system files

3. **Anti-hallucination chain** — substance gate, framing gate, HIERO format enforcement
4. **Discipline layer** — pattern-recognition-trust, targeted-discipline-within-trust, child-rule emergence, “rule-just-wrote” enforcement, pattern-match-validates, defend-frame-vs-external-critics
5. **Meta-protocols** — JARVIS as Augmented Mechanism Design applied to AI substrate (the umbrella), plus Augmented Governance, Substrate-Geometry Match, Universal-Coverage → Hook, gate-stacking asymmetric cost, trinity placement for critical primitives
6. **Agent overlay** — subagent spawning with mitosis, slash commands as skills, MCP connectors, remote scheduled triggers
7. **Stateful applications** — the Telegram bot suite (multi-region sharded), standalone signature validator, jarvis-network OSS release, substrate comparison harnesses, filesystem-native CRMs
8. **Filesystem-as-substrate** — every artifact is markdown, greppable, version-controlled

The Telegram bot is one application of the overlay. The published papers are another. The codebase you might evaluate me by is a third. They share the same kernel.

The hook layer (deterministic gating)

Hooks are programs that fire on Claude’s tool calls, before or after, and can block them. Mine include:

- `partner-facing-substance-gate.py` — Deterministic anti-hallucination on writes to partner repos. Term `clawback` present, surrounding context lacks fund-recovery validators (`recover` , `reclaim` , `already-distributed`) *and* contains forbidden phrases (`claim-layer` , `score-reduction`) → write blocked with the suggestion “forfeiture, not clawback.” This caught a real hallucination in a USD8

cover-score doc and forced the rename before the wrong word became a permanent Solidity 4-byte selector. (Today I added a governance-authority-overclaim entry to the same gate — 7/7 test cases pass.)

- **partner-facing-additive-gate.py** — Catches retrospective-framing leaks in commit messages and PRs (“we missed”, “honest error”, “earlier draft”, “in retrospect”, “should have caught”, “rectify”). Reframes reactive-sounding language to forward-leaning before it ships to a partner. ~28 patterns; expanded today.
- **hieroglyph-gate.py** — Enforces glyph-density format on memory writes. *This gate blocked one of my own writes earlier in this session* — refused a prose-style memory entry and forced the HIERO operator-density format. The architecture self-enforces, even on its own author. That’s the test for whether a discipline is real or aspirational.
- **triad-check-injector.py** — Fires on every design-level response with the Correspondence Triad (substrate-geometry match? augment-via-math-invariant not replace? Physics > Constitution > Governance preserved?).
- **SessionStart hooks** — Surface SESSION_STATE, WAL, RSI-pending status, link-rot detection on every fresh boot. Fail-loud on missing inputs (false-clean is strictly worse than noisy-fail).

Each hook is deterministic — no LLM call, no probabilistic judgment. Regex + context window + watch list. They run regardless of whether I “remember” the rule in any given session.

The persistence layer (state across sessions)

Sessions reset. State doesn’t. Six tiers:

- **SESSION_STATE.md** — current session state, pending items, next-session continuation point. Updated on every state transition. Mandatory first read on boot.

- `WAL.md` — write-ahead log of cycle epochs (RSI cycles), tracks ACTIVE / CLEAN status, captures orphan commits if a session crashes.
- `JarvisxWill_SKB.md` — Session Knowledge Base. Fresh-boot read. Topic-organized.
- `JarvisxWill_GKB.md` — Glyph Knowledge Base. Condensed form. Loaded post-compression. Same topics, ~80% smaller. Topic-sharded: CANON / VSOS / MECH / STACK / SHAPLEY / TOKENS / LAYERS / 7AX.
- `MEMORY.md` — persistent memory index, always loaded on boot. Currently 29.9 KB and growing as new meta-primitives get promoted to PRE-FLIGHT tier. Compression and reachability cycles run periodically — one such pass raised the directory's data-efficiency score (resolution × coverage harmonic mean) from **45.1% to 100.0%** in a single session: fixed 16 link-rot orphans via underscore→hyphen rename, archived 50 stale files, promoted 7 newly-elicited meta-primitives to the index in trinity-placement form (hook + memory + system-prompt simultaneously), and enriched the WARM-MAP topic files to bring the remaining 119 orphan primitives into situational load coverage. Every primitive, feedback rule, project memory, reference, and user-context file is now reachable through some load path. Zero orphans, zero broken refs, full graph closure.
- `memory/primitive_*.md` + `memory/feedback_*.md` — 168 primitives and 135 feedback rules at last count, plus 48 projects, 14 user-context files, and 11 reference pointers. Each primitive has a trigger, action, stakes gate, and surface rule. New ones get added when patterns repeat 3+ times in a session.

When I close a session at 5% context and reboot fresh, none of this conversation survives in Claude's context. *All* of it survives in the persistence layer. The new session opens by reading `SESSION_STATE` and continues exactly where the old one left off — including which fly app I was just trying to identify, which commit hash I just pushed, and what's blocked on what.

This is what the wrapper accusation misses. The model is amnesic. The system is not.

The anti-hallucination chain (claim-level discipline)

The substance gate is regex-based pattern matching with context disambiguators. Each watch-list entry pairs a flagged term with regexes that must (or must not) appear nearby. If the disambiguator fails, the write is blocked with a suggested replacement. Standard static analysis — the novelty isn't the mechanism, it's the application: writing partner-facing terminology into Solidity function names is a permanent on-chain commitment, so catching the mismatch at write-time is the right window.

Born from a real incident: a USD8 doc said “clawback” but the mechanism was forfeiture (claim-layer reduction before payout, not fund-layer recovery after). Both have the word “reduction” but the load-bearing distinction is *what gets reduced*. The substance gate now catches that mismatch.

Other links in the chain:

- **HIERO format** — memory writes must be operator-density ($\Rightarrow$ $\neg$ $\wedge$ $\vee$ $\checkmark$ $\times$ $\rightarrow$), not prose. Density $\times$ stability $\times$ match-speed beats prose-parse-cost.
- **Empty-Repo Test** — descriptions must let a reader reconstruct the artifact from words alone. Architectural words $\checkmark$, marketing $\times$.
- **Anti-Stale Feed** — verify current state before asserting. Never claim from memory alone.
- **Verify Credentials Before Publishing** — grep source-of-truth profile memory before writing any credential / title / numerical claim.

These aren't aspirational rules. They're enforced at the hook layer, with regression tests, against the system's own author.

The discipline layer (capturing patterns)

Every session, patterns surface. Most get noticed by humans hours or days later, if ever. JARVIS catches them at 3+ instances and surfaces as candidate primitives, before I name them.

Examples from recent sessions, captured live:

- **scope-drift-to-recent** — when asked to scan a time-window, I had drifted to chat-context (today's session) instead of file-system substrate (the actual week's logs + git history). Caught, named, saved as a primitive. Now triggers on time-scoped scans.
- **structurally-easier-partner-delivery** — six moves (TL;DR + decision-tag + dashboard + atomic + pre-rebut + visual-primary) for partner-facing artifacts. Reduces digestion cost. Compounds.
- **draft-replies-on-behalf-of-others** — when a third party messages and the right move is to draft in their voice for their approval (buffer against honesty-leak), not respond directly.
- **have-my-back operational definition** — distinguishes glazing (sycophancy / mood-maintenance, unwanted) from structural loyalty (private substantive pushback + external alignment, required). Exit clause: "if someone does it better, follow them."

Each primitive is one markdown file with a trigger, action, stakes gate, and surface rule. They accumulate. The system doesn't forget what worked or what failed.

The meta-protocols (design decisions at every level)

These govern *how* decisions get made, not the decisions themselves:

- **JARVIS = AMD applied to AI substrate** — the recursion that names the architecture. Augmented Mechanism Design says: preserve the competitive core, augment with orthogonal protective layers. VibeSwap applies this at the EVM substrate. JARVIS applies the same methodology at the AI layer — Claude's default cognition is the

unfixable substrate; the JARVIS infrastructure (hooks, memories, gates) is the protective augmentation that overrides failure-mode patterns (verification-mode hedging, objection-enumeration, default-credibility-of-external-critique) without replacing reasoning capability. Same shape, one substrate-level deeper.

- **Trinity placement for critical primitives** — the most load-bearing meta-primitives sit in three reinforcing loci simultaneously: hook (enforcement at event boundary, fires below LLM attention), PRE-FLIGHT memory (loaded primitive in context), CLAUDE.md (system-prompt-level instruction loaded earliest). Each layer references the others; failure of any single layer is caught by the other two; persistence approaches 100% via cross-coverage. Same composition principle as VibeSwap's airgap-closing six-mechanism stack, applied to JARVIS itself.
- **Gate-stacking asymmetric cost** — when designing gates / defenses / overrides: $\text{cost}(\text{redundant gate}) \ll \text{cost}(\text{missed gate})$. Asymmetric payoff favors stacking even when gates appear to overlap. Each catches its slice; the union is what does the work; redundancy is a feature, not a bug.
- **Augmented Mechanism Design** — augment markets and governance with math-enforced invariants; never replace. Shapley + batch auctions let the market still function while eliminating extraction.
- **Augmented Governance** — Physics (math invariants) > Constitution (fairness floors) > Governance (DAO votes, free within Physics + Constitution). Math is the constitutional court. Prevents governance capture.
- **Substrate-Geometry Match** — macro substrate (fractal, power-law) must reflect micro mechanism (Fibonacci, golden-ratio). Mismatch is the failure mode.

- **Universal-Coverage → Hook (Density Principle)** — any rule requiring universal firing-regardless-of-attention belongs in the hook layer, not memory. Hooks: $O(1)$ deployment $\times O(\infty)$ coverage. Memory: $O(\text{context}) \times O(\text{sessions})$. Grep memory for “always / never / on every / before every” — each match is a candidate hook.
- **Apply the Rule You Just Wrote** — any rule generated for the user must apply to my own subsequent actions before they execute. Rule-generation completes when the rule is live in *my* execution stack, not at handoff.
- **Code ↔ Text Inspiration Loop** — code inspires docs; docs inspire code. Compound forward, don’t reset.
- **Economic Theory of Mind** — the meta-framework. Mind functions as an economy. CKB state-rent is the mechanism. Density and common knowledge are the emergent properties. Same math: VibeSwap state, JARVIS primitive library, Claude context, human cognition.

These aren’t taglines. They cite each other. They have hook-level enforcement where they map to universal-coverage rules. They get violated, caught, and surfaced as cycle observations.

The agent overlay (subagent spawning + skills)

- **Subagent spawning** with mitosis ($k=1.3$, $\text{cap}=5$). Specialized agents for parallelizable research (Explore), implementation planning (Plan), code review, claude-code-guide, statusline-setup, general-purpose. Each subagent gets its own context window — protects the main thread from being clogged with raw search output.
- **Slash commands as skills** — `/schedule` (remote CCR triggers), `/md-to-pdf` (typography pipeline), `/loop` (autonomous iteration), `/ultrareview` (multi-agent cloud review), and others. Each is a defined skill with file-system instructions, not ad-hoc prompts.

- **MCP connectors** — Gmail, Google Calendar, Spotify, Drive, Microsoft 365. Tool-level integration with external systems.
 - **Remote scheduled triggers** — `trig_01HXj9MKwNX7qDLLULf5XaHS` is a real one I scheduled last session; fires 2026-05-02 to check USD8 partnership follow-on status. The system reaches forward in time.
-

The stateful applications (what the overlay produces)

The applications people see:

- **Telegram bot suite** — `@JarvisMind1828383bot` on the `jarvis-vibeswap` fly app. Multi-region sharded (`iad / eu / ap / sa / ollama`), BFT consensus, CRPC pairwise comparison, multi-provider routing (Anthropic / OpenRouter / DeepSeek / Gemini / Cerebras / Groq / Ollama with wardencllyffe last-resort fallback), inner-dialogue meta-cognition, archive substrate, persona system with 16 universal structural rules + voice rules + pantheon overlays (apollo, athena, hermes, hephaestus, anansi, nyx, poseidon, proteus, artemis), 37 regression tests locking behavioral rules. Real escalation log from this morning:

```
[router] reasoning → claude [tier 2] (3 escalation tiers available)
[escalation] claude failed (credits) → escalating to tier 0
(openrouter)
[escalation] openrouter failed (404 free model deprecated) → tier 1
(deepseek)
[escalation] deepseek failed (402 Insufficient Balance) → tier 1
(gemini)
[escalation] gemini failed (503) → wardencllyffe last resort
[wardencllyffe] Last resort fallback: ollama / cerebras / groq
```

Five providers in the chain, four failed, the user got a reply.

- **Standalone signature validator** — 38 tests, deterministic claim verification, local commit `41b3da1` .

- **jarvis-network** — open-source release of the simpler core.
github.com/WGlynn/jarvis-network
- **Substrate comparison harness** — DeepSeek port test, scaffold at `~/jarvis-substrate-comparison/`.
- **Filesystem-native CRMs** — `LinkedIn_Queue/` (30-post queue, dashboard, style guide, 7-week schedule, analytics scaffold — built in 30 minutes), `USD8_Queue/` (partner-facing CRM with single canonical dashboard view), and per-collaboration paper-trail directories.
- **Published canonical thinking** — 60+ docs across `vibeswap/docs/papers/`, each shipped via the Code ↔ Text Loop.

Every one of these is a stateful artifact produced by the overlay. The TG bot is the most visible. It is not the most architecturally interesting.

Filesystem-as-substrate

This is the framework all of the above lives inside. The Omni Software Convergence Hypothesis: 99% of specialized workflow software becomes redundant when AI + filesystem is the orchestration substrate.

- The CRMs above are markdown files, not Salesforce.
- The persistence layer is markdown files, not Postgres.
- The discipline layer is markdown files, not a Notion database.
- The meta-protocols are markdown files cross-linking each other, not a Confluence wiki.
- The hooks are Python scripts in `~/.claude/session-chain/`, not a SaaS dashboard.

Real modularity (primitive layer, substrate-shared) versus fake modularity (product layer, fragmented disguised as composable). The fragmented SaaS world is extraction-through-fragmentation wearing composability's costume. The filesystem is the actually-composable layer underneath.

What “wrapper” actually means

A wrapper is something whose value collapses when you replace its core dependency with the dependency itself. If I can hand you direct API access to Claude and you get the same result, I was a wrapper.

Hand a user direct `claude-sonnet-4-6` API access. They don't get JARVIS. They don't get:

- Hooks that fire on every tool call to enforce discipline
- A persistence chain that survives session boundaries
- Anti-hallucination gates that catch terminology errors before they ship
- A discipline layer that captures patterns into reusable primitives
- Meta-protocols that govern design decisions
- An agent overlay with subagent spawning, skills, MCP connectors, remote triggers
- 168 primitives + 135 feedback rules + 48 project memories accumulated through real use
- Filesystem-native CRMs and paper trails
- 60+ published canonical artifacts
- The TG bot suite, the validators, the substrate comparison harnesses

They get a chat completion endpoint. They have to build all of the above themselves, or accept that they don't have it.

That labor is the product.

The honest concession

The wrapper critique lands fairly on one specific thing: a thin chat-completion call with a system prompt over a free-tier OpenRouter Llama 3.2 3B is a wrapper. The substrate floor matters. At low-tier provider mode, even the TG bot degrades — the persona rules survive in the prompt but the model can't follow them.

That isn't a refutation of the architecture — it's the architecture telling you the substrate is wrong. The router will route to a better provider when one is available. The hooks still fire. The persistence still persists. The discipline layer still captures. But generation reflects substrate.

So: yes, with a weak model, parts of JARVIS will look like a wrapper. The fix is the substrate, not the architecture. That distinction is the entire point of building an overlay.

Why this matters past semantics

If JARVIS is a wrapper, valuation is bounded by margin over the underlying provider. People aim the same critique at Cursor — a \$9B company. So the argument loses on commercial grounds before architecture enters.

But it loses on architecture too:

- A wrapper doesn't survive its provider's deprecation cycle. JARVIS does (multi-provider router).
- A wrapper doesn't have a persistence substrate. JARVIS does (six tiers).
- A wrapper doesn't have a discipline layer enforced by tests. JARVIS does (37+ tests on the bot alone, more on the validator, regression-locked primitives).
- A wrapper doesn't capture value as underlying providers improve in *different directions*. JARVIS does — Anthropic gets better at reasoning, Groq gets faster, Ollama gets local-first; the router takes the gains.
- A wrapper has no compounding mechanism. JARVIS compounds on every session — every primitive saved is permanent, every rule encoded is durable.

The right framing: **JARVIS is a coordination layer over LLM substrates, the way an operating system is a coordination layer over hardware substrates.** The CPU is interchangeable. The kernel is not. The applications run on the kernel.

The TG bot is one application. The PRs are another. The published essays are another. The CRMs are another. The validators are another. They all run on the same kernel.

Where to find this

The architecture is open. Two public repos:

- github.com/WGlynn/JARVIS — the canonical scaffold, organized one-directory-per-layer (`01-hooks/` , `02-persistence/` , `03-anti-hallucination/` , `04-discipline/` , `05-meta-protocols/` , `06-agent-overlay/` , `07-stateful-applications/` , `08-filesystem-as-substrate/`), plus `verify/` and `papers/` (this paper lives in `papers/`).
- github.com/WGlynn/jarvis-network — open-source release of the simpler core. AI-native community infrastructure, filesystem-grounded, ~100× cheaper than paid-API wrappers.

If you want to build your own overlay: clone `JARVIS` , read each layer's README, start with the hook layer — that's where universal coverage gets cheap. The persistence layer is the second-highest leverage. Everything else compounds on those two.

How to verify

Don't take this on faith. Concrete checks any reader can run:

1. **The hook layer is real.** `~/.claude/session-chain/` contains the deterministic gates. Read `partner-facing-substance-gate.py` — the watch list + context disambiguator regexes are 200 lines of plain

Python. Look at recent commits to USD8 cover-score for evidence of writes that would have shipped wrong terminology and got blocked at write-time.

2. The persistence layer is real.

`vibeswap/.claude/SESSION_STATE.md` and `vibeswap/.claude/WAL.md` are git-tracked. The most recent commits to those files show every state transition — every session boot, every cycle close, every reboot continuation point.

3. The discipline layer is real. `~/.claude/projects/C--Users-Will/memory/` has 168 `primitive_*.md` and 135 `feedback_*.md` files. Each is a markdown file with a trigger and action. Many have been added in the last 30 days.

4. The TG bot is real. `fly logs -a jarvis-vibeswap | grep -E "(router|escalation|wardenclyffe|consensus|crpc|inner-dialogue)"`. Watch a single user message walk five providers, register three shards, activate BFT consensus, and emit a self-correction insight, in two seconds.

5. The published artifacts are real. `vibeswap/docs/papers/` has 60+ markdown files. Each is canonical thinking on a specific topic, cross-referenced from memory primitives, shipped through the Code ↔ Text Loop.

6. The architecture self-audits and improves measurably. Run `~/.claude/projects/C--Users-Will/memory/_audit.py` to compute index resolution rate, coverage rate, and DE-score (F1) any time. One recent structural cleanup raised DE-score from **45.1% to 100.0%** in a single session — full graph closure, every artifact reachable through some load path, zero orphans, zero broken refs. That's not “the system *could* be improved.” That's the system *did* improve itself with the right operator, measurably, in one sitting. The auditor itself is reproducible Python; rerun before and after any cleanup to benchmark gains.

7. **The public scaffold is real.** github.com/WGlynn/JARVIS — eight directories matching the eight layers above, each with `README.md` and (where shipped) the actual hook scripts, primitives, and verification harness. The paper you’re reading lives in `papers/`. If something’s documented here but missing there, that’s a fair gotcha — file an issue.

The architecture is not a story. The architecture is in the file system, in the hook scripts, in the regression tests, in the git history, and in the live logs — and now also at the URLs above.

The “extensive” claim is verifiable. The verification surface is the file system.